



Documento de Diseño de Videojuego (GDD) Profesional

Integrantes	- Benjamín Matías Pavez Vidal - Nicolas Renato Ramírez Berríos
Asignatura	Programación de Videojuegos
Docente	Leopoldo Esteban Rodriguez Bravo

Iquique – Chile

[25-05-2026]

Índice

1. Historial de Revisión	3
2. Introducción y Alcance	4
2.1. Concepto Principal (High Concept)	4
2.2. Convenciones de Estilo	4
3. Especificaciones del Sistema (Target System)	5
3.1. Hardware y Software	5
4. Mecánicas de Juego y Objetivos	5
4.1 Modos de Juego	5
5. Inteligencia Artificial y Comportamiento del Mundo	6
5.1. Lógica de Enemigos	6
5.2. Interactividad del Entorno	6
6. Arquitectura Técnica y Estructuras de Datos	7
6.1. Diagrama de Clases	7
6.2. Máquina de Estados (FSM)	9
7. Herramientas de Desarrollo y Edición	12
7.1. Editor de Niveles	12

1. Historial de Revisión

Versión	Autor	Fecha	Comentarios / Cambios
1.00	Benjamín Pavez Nicolás Ramírez	20/04	Planteamiento inicial del juego con el primer nivel creado, un enemigo, animaciones del personaje, hud, menú de pausa, música de fondo, background continuo que se mueve con el player, animaciones completas para el player.
2.00	Benjamín Pavez Nicolás Ramírez	18/05	Se implementó efectos de sonido, efectos visuales, menú principal, 3 nuevos enemigos, un nuevo nivel, pantalla de carga, NPCs y nuevas funciones como el marcador de tiempo y el minimapa.

2. Introducción y Alcance

2.1. Concepto Principal (High Concept)

Unas plataformas en 2D que puede jugarse en solitario de forma tradicional, *pero que brilla en su modo multijugador Cooperativo: donde el jugador 2 ayuda al jugador 1 con la opción de colocar nuevas estructuras (aun en discusión) mejoras y la capacidad de moverse y atacar libremente al poseer enemigos.*

2.2. Convenciones de Estilo

Este documento sigue las convenciones estándar de un GDD profesional. A continuación, se detallan las reglas de formato aplicadas en cada sección.

- **Convenciones de Texto**

Texto normal: se utiliza para describir características ya confirmadas e implementadas en el juego.

Texto en cursiva: se reserva para ideas propuestas, funcionalidades en discusión o elementos pendientes de implementación.

- **Nomenclatura de Scripts y Clases**

PascalCase para nombres de clases y MonoBehaviours (ej.: *PlayerControllerComplete*, *EnemyBase*). **camelCase** para variables privadas y parámetros (ej.: *currentHealth*, *jumpForce*). **UPPER_SNAKE_CASE** para constantes (ej.: *KEY_VOLUME*).

- **Convenciones del Motor (Unity)**

Los GameObjects en la jerarquía siguen el patrón: [Categoría]_[Nombre] (ej.: *Enemy_Knight*, *VFX_Slash*, *UI_HUD*).

Las escenas están numeradas por índice de build para facilitar la navegación desde código (`SceneManager.LoadScene`). La escena 0 corresponde siempre al Menú Principal.

Las variables expuestas al Inspector se documentan con [Header] y [Tooltip] para facilitar el trabajo en equipo.

3. Especificaciones del Sistema (Target System)

3.1. Hardware y Software

- **Plataforma:** PC (Windows).
- **Motor de Juego:** Unity
- **Resolución Base:** Pixel Art. Para mantener la estética retro del personaje y adaptarse al formato panorámico (16:9) de los monitores actuales, se utilizará una resolución base de 1920x1080 y se podrá cambiar la resolución en el menú para la resolución 1280x720.

4. Mecánicas de Juego y Objetivos

4.1 Modos de Juego

- **Modo Individual:** Es el modo principal y actualmente implementado. El jugador recorre el nivel en solitario con el objetivo exclusivo de llegar a la meta, buscando superar constantemente su propio tiempo récord.
- **Modo Multijugador Cooperativo (Online):** Variante cooperativa en la cual el segundo jugador podrá ayudar al primero colocando estructuras o mejoras para el otro jugador y podrá controlar a los enemigos para apoyarlo.

4.2 Acciones del Jugador

- **Movimiento horizontal:** Teclas **A** (Izquierda) y **D** (Derecha).
- **Agacharse:** Tecla **S**.
- **Salto Básico:** Tecla **W** o **Barra Espaciadora**.
- **Doble Salto:** Acción que se activa al presionar nuevamente el botón de salto mientras el player está en el aire.
- **Salto de Pared (Wall Jump):** Capacidad de rebotar y ganar altura al presionar el botón de salto mientras el player hace contacto con una superficie vertical.
- **Dash (Impulso):** Movimiento rápido de evasión o reposicionamiento usando la tecla **Shift** o el **Clic Derecho** del ratón.
- **Ataque:** Golpe o acción ofensiva principal utilizando el **Clic Izquierdo** del ratón o la tecla **J**.
- **Comportamiento de Cámara:** La cámara no es libre; se mantiene fija siguiendo en todo momento al player.
- **Interacción específica:** Capacidad de interactuar con objetos o con npcs utilizando la tecla **E**

5. Inteligencia Artificial y Comportamiento del Mundo

5.1. Lógica de Enemigos

- **Estado de Reposo (Standby):** Los enemigos permanecen inactivos en sus posiciones iniciales mientras no haya amenazas cercanas.
- **Detección y Persecución:** Cada enemigo posee un área de detección (trigger) definida. Cuando el jugador ingresa a este radio, el enemigo cambia su estado a agresivo y comienza a acercarse hacia el avatar con el objetivo de atacar y detener su avance.

5.2. Interactividad del Entorno

- **Trampas Estáticas:** Por el momento, el nivel cuenta con antorchas fijas ubicadas estratégicamente en el suelo.
- **Daño por Contacto:** Antorchas las cuales son elementos hostiles. Cada vez que el jugador entra en contacto o colisiona con una de ellas, sufre daño por quemadura.

6. Arquitectura Técnica y Estructuras de Datos

6.1. Diagrama de Clases

Clase	Responsabilidad
Entidades	
<i>PlayerControllerComplete</i>	Controlador principal del jugador. Gestiona movimiento horizontal, salto doble, wall-jump, dash con cargas, agacharse, ataque cuerpo a cuerpo, vida, knockback y respawn.
<i>PlayerSoundController</i>	Gestiona todos los efectos de sonido del jugador (ataque, daño recibido, salto, dash, poder y muerte) mediante AudioSource.PlayOneShot.
<i>PlayerEffectsController</i>	Gestiona los VFX del jugador: partículas de polvo al saltar y sprite de tajo al atacar, con escala invertida según la dirección.
<i>DestroyVFX</i>	Componente auxiliar que autodestruye un GameObject de efecto visual tras un tiempo configurable (lifetime).
<i>EnemyBase</i>	Clase base abstracta de todos los enemigos. Centraliza vida, daño por contacto, knockback, stun, animación de muerte y fade-out.
<i>EnemyFollow</i>	Enemigo terrestre que persigue al jugador dentro de un radio de detección, con detección de borde para evitar caídas. Daña por contacto.
<i>EnemyLongSwordKnight</i>	Enemigo terrestre con IA de espadón: persigue al jugador y ejecuta un ataque en área con tiempos de anticipación y recuperación configurables.
<i>EnemyFlyingShooter</i>	Enemigo volador que mantiene distancia, se desplaza con oscilación sinusoidal y dispara proyectiles en la dirección del jugador.

<i>EnemyProjectile</i>	Proyectil disparado por <i>EnemyFlyingShooter</i> . Se mueve en línea recta, daña al jugador al impactar y se autodestruye por tiempo o colisión.
Managers y Sistemas	
<i>GameManager</i>	Singleton central. Controla pausa, contador de tiempo (con penalizaciones y recompensas mediante <i>ModificarTiempo</i>), condición de victoria, guardado-y-salida, y navegación de escenas. Al completar el nivel, llama a <i>RecordSystem.TrySetRecord</i> para registrar o actualizar el mejor tiempo.
<i>SaveSystem</i>	Singleton de persistencia. Serializa y deserializa <i>SaveData</i> a JSON en disco. Persiste entre escenas con <i>DontDestroyOnLoad</i> .
<i>LoadingScreenManager</i>	Singleton que gestiona la carga asíncrona de escenas con barra de progreso suavizada (fake progress).
<i>MainMenuManager</i>	Controla el menú principal: nueva partida, continuar partida guardada, opciones, scoreboard y salida. Muestra el botón Continuar solo si existe guardado. Gestiona el panel Scoreboard instanciando filas dinámicas con los datos de <i>RecordSystem</i> (nivel y mejor tiempo).
<i>OptionsManager</i>	Gestiona configuración de volumen (<i>AudioListener</i>) y resolución de pantalla con persistencia en <i>PlayerPrefs</i> . Activo en menú y pausa.
<i>SaveData</i>	Modelo de datos serializable. Almacena: índice de escena, posición del jugador, vida, tiempo transcurrido y lista de enemigos vivos.
<i>RecordSystem</i>	Singleton de persistencia de récords por nivel. Serializa y deserializa <i>AllRecords</i> (lista de <i>LevelRecord</i>) en <i>PlayerPrefs</i> como JSON. Expone <i>TrySetRecord</i> para registrar o actualizar el mejor tiempo, y <i>GetAll</i> para listar todos los récords (usado por el Scoreboard del menú principal).

<i>LevelRecord</i>	Modelo de datos serializable que representa el récord de un nivel. Almacena: índice de escena (sceneIndex), nombre de la escena (levelName) y mejor tiempo registrado (bestTime en segundos).
Minimap	
<i>MinimapController</i>	Controla el minimapa en pantalla. Mueve la cámara secundaria del minimapa siguiendo al jugador con interpolación suave (Vector3.Lerp). Permite expandir/contraer el panel con la tecla Tab (alterna entre tamaño pequeño y grande) y se pausa cuando el GameManager está en pausa.
UI y Entorno	
<i>Dialogue</i>	Sistema de diálogo con tipeo progresivo. Bloquea los controles del jugador (isTalking) mientras dura la conversación con un NPC.
<i>CameraFollow</i>	Cámara suave que sigue al jugador en LateUpdate con Vector3.SmoothDamp y un offset configurable.
<i>DeathZone</i>	Trigger que aplica 999 de daño al jugador o al enemigo que caiga fuera del mapa, garantizando muerte instantánea.
<i>LevelGoal</i>	Meta del nivel. Puede requerir que un jefe esté muerto antes de activarse. Al desbloquearse llama a GameManager.WinLevel().
<i>Parallax</i>	Desplaza la textura UV de un Quad según el movimiento horizontal de la cámara, creando efecto de profundidad.
<i>ColumnParallax</i>	Variante de Parallax que además sigue la posición de la cámara y soporta densidad de textura configurable. Se pausa con el juego.

6.2. Máquina de Estados (FSM)

Jugador (PlayerControllerComplete)

Estado	Condición de Entrada	Descripción / Comportamiento
--------	----------------------	------------------------------

Idle	horizontalInput == 0 y isGrounded	El jugador está quieto en suelo. Animator: movement=0, IsGrounded=true.
Walk	horizontalInput != 0 y isGrounded	Movimiento a walkSpeed. Puede entrar en Crouch o iniciar Dash.
Crouch	Tecla S pulsada y isGrounded	Velocidad reducida al 50%, collider reducido al 55% de altura.
Jump	Espacio/W presionado, jumpsRemaining > 0	Aplica impulso vertical. Activa holdForce mientras se mantiene la tecla hasta jumpHoldMaxTime.
DoubleJump	Espacio/W en aire, jumpsRemaining == 1	Segundo salto disponible. Trigger 'DoubleJump' en Animator.
WallSlide	isTouchingWall y !isGrounded y input hacia la pared	Limita velocidad de caída a wallSlideSpeed durante wallSlideDuration.
WallJump	Espacio pulsado durante WallSlide	Lanza al jugador en dirección contraria a la pared. Bloquea input horizontal durante wallJumpDuration.
Dash	Shift / Clic Derecho, charges > 0	Impulso instantáneo a sprintSpeed durante sprintDuration. Consume una carga; se recarga con cooldown.
Attack	Clic Izq / J, Time.time >= nextAttackTime	OverlapCircle en attackPoint. Daña EnemyBase dentro del rango. Cooldown: 0.5 s.
Hurt	TakeDamage() invocado	Knockback + trigger 'Hurt'. El jugador pierde control por knockbackDuration.

Dead	currentHealth <= 0	isDead = true. Velocidad = 0. Tras 1.5 s ejecuta Respawn() restaurando vida y posición.
Talking	isTalking == true (NPC activo)	Bloquea todo input de movimiento y combate. La cámara y las animaciones siguen activas.

Enemigo Seguidor (EnemyFollow)

Estado	Condición de Entrada	Descripción / Comportamiento
Idle	distanceToPlayer >= detectionRadius	El enemigo permanece inmóvil. Velocidad horizontal = 0.
Chase	distanceToPlayer < detectionRadius	Se mueve hacia el jugador a speed. Verifica suelo adelante para evitar caídas.
Stop	distanceToPlayer <= stoppingDistance	Se detiene frente al jugador. Contacto activa OnCollisionEnter2D y aplica contactDamage.
Stunned	TakeDamage() recibido	isStunned = true durante stunDuration. No se mueve. Aplica knockback.
Dead	health <= 0	isDead = true. Collider desactivado. Fade-out y Destroy tras deathDelay segundos.

Caballero Espadón (EnemyLongSwordKnight)

Estado	Condición de Entrada	Descripción / Comportamiento
Idle	distanceToPlayer >= detectionRadius	Enemigo quieto, velocidad = 0.

Chase	distanceToPlayer < detectionRadius, jugador fuera de alcance	Se mueve a speed. Opcionalmente detecta bordes con raycast.
Attack (Prep)	OverlapCircle detecta jugador en attackRange	Detiene movimiento, lanza trigger 'Attack'. Espera attackAnticipationTime.
Attack (Hit)	Tras anticipación, cooldownTimer <= 0	OverlapCircle aplica swordDamage al jugador. Espera attackRecoveryTime.
Stunned	TakeDamage() recibido	isStunned = true. Se limpian triggers de animación. No se mueve.
Dead	health <= 0	isDead = true. Collider desactivado. Fade-out y Destroy.

Ángel Volador (EnemyFlyingShooter)

Estado	Condición de Entrada	Descripción / Comportamiento
Patrol (Float)	distanceToPlayer >= detectionRange	Flota con movimiento sinusoidal en Y mediante linearVelocity. Sin desplazamiento horizontal.
Chase	distanceToPlayer < detectionRange	Se mueve hacia posición objetivo (jugador - followDistance) con oscilación vertical.
Shoot	fireTimer <= 0 (cada fireCooldown segundos)	Instancia EnemyProjectile orientado hacia el jugador. Trigger 'Attack' en Animator.
Stunned	TakeDamage() recibido	Congela linearVelocity = 0. Activa flash rojo (FlashRoutine). isStunned durante stunDuration.
Dead	health <= 0	gravityScale = 1 (caída física). Collider desactivado. Fade-out y Destroy.

7. Herramientas de Desarrollo y Edición

7.1. Editor de Niveles

Los escenarios del juego se construyeron íntegramente con el sistema de Tilemap de Unity, aprovechando el conjunto de herramientas 2D que el motor ofrece de forma nativa.

Sistema Tilemap

Cada nivel utiliza múltiples capas de Tilemap superpuestas para separar elementos visuales y de colisión. La estructura típica por nivel es la siguiente:

- **Tilemap – Suelo / Plataformas:** capa principal de colisión. Tiene asignado un *Tilemap Collider 2D* con la opción *Composite Collider* activada para optimizar la física.
- **Tilemap – Decoración / Fondo:** capa visual sin colisión. Contiene adornos, paredes de fondo y elementos del escenario.
- **Tilemap – Peligros:** tiles especiales con la etiqueta "*Damage*" que causan daño por contacto (ej.: antorchas, pinchos).

Herramientas Utilizadas

- **Tile Palette:** ventana principal de edición. Permite seleccionar, pintar, borrar y rellenar tiles directamente en la vista de escena. Se utilizó el modo *Rect* para pintar áreas grandes de suelo y el modo *Single* para detalles finos.
- **Rule Tile:** tipo de tile inteligente que selecciona automáticamente el sprite correcto según los tiles vecinos. Se configuró para el suelo principal, logrando transiciones naturales entre esquinas, bordes y relleno sin intervención manual.
- **Sprite Atlas:** todos los tiles de un mismo tema visual se empaquetan en un atlas para reducir el número de draw calls y mejorar el rendimiento.
- **Grid / Grid Layout:** objeto raíz que contiene todos los Tilemaps del nivel. Define el tamaño de celda estándar (1×1 unidades Unity), garantizando coherencia con las colisiones y el pixel art.
- **Tilemap Collider 2D + Composite Collider 2D:** combinación usada en la capa de colisión. El Composite fusiona los colliders individuales de cada tile en una

única malla, eliminando micro-colisiones internas y mejorando la estabilidad del Rigidbody del jugador.

Flujo de Trabajo

El proceso de construcción de un nivel sigue estos pasos: (1) se diseña el layout del nivel en papel o boceto digital; (2) se pinta la capa de colisión con el Rule Tile del suelo; (3) se añaden capas de decoración y detalles visuales; (4) se colocan Prefabs de enemigos, puntos de spawn y la meta del nivel; (5) se ajustan las zonas de DeathZone en los bordes del mapa.